CampusOS

Portal web pentru studenți

Administratorul editează orarul comun, publică note și adaugă taxe de cămin. Studentul își consultă datele, își gestionează examenele și termenele și își rezervă un interval la spălătorie.

De la browser la baza de date

01 FRONTEND

HTML și CSS

Flask completează șabloanele Jinja cu datele afișate în browser.

`templates/ și static/`

→

02 BACKEND

Python și Flask

Primește formularele și verifică rolul și regulile aplicației.

`app.py și campus.py`

→

03 BAZĂ DE DATE

MySQL 8.4

Păstrează conturile, orarul, notele, taxele și rezervările.

`schema.sql și db.py`

Exemplu / Administratorul adaugă o oră

01	**Formular**	În browser, adminul introduce materia, ziua și intervalul.
02	**Permisiune**	Flask verifică rolul înainte să accepte modificarea.
03	**Regulă**	Funcția din planner.py verifică suprapunerile din ziua aleasă.
04	**Salvare**	Dacă intervalul este liber, PyMySQL salvează ora în MySQL. Studentul o vede în orarul comun.

Baza de date se creează din scripturi

La pornire, Flask execută **schema.sql** și creează tabelele lipsă. Aplicația folosește interogări parametrizate prin PyMySQL. La spălătorie, o constrângere unică permite o singură rezervare pe interval.

Rulare cu Docker și Linux

Docker Compose pornește serviciile **app** și **db**. Browserul se conectează la aplicația Flask prin portul configurat pe calculator.

Mediul folosit la testare

Proiectul a fost pornit din Windows cu Docker Desktop și WSL instalat. Flask și MySQL rulează în containere Linux. Separat, testele aplicației au rulat pe Linux cu MySQL 8.4 și au verificat autentificarea, accesul la note și rezervările simultane.

Ce face Docker Compose

`db`	Pornește MySQL 8.4. Aplicația așteaptă ca baza să accepte conexiuni.
`app`	Construiește imaginea din Dockerfile, aplică schema SQL și pornește Flask.
`mysql_data`	Păstrează datele MySQL după oprirea și recrearea containerelor.

Comenzi pentru demonstrație

Rulează comenzile din folderul cu **compose.yaml**, după ce ai configurat **.env**. Lasă fereastra de pornire deschisă și rulează verificările în alt terminal.

`pwd && ls`	Verific folderul și fișierele din shell-ul Linux.
`docker compose up --build`	Construiesc și pornesc aplicația.
`docker compose ps`	Verific starea containerelor și portul.
`docker compose logs -f app`	Urmăresc mesajele aplicației.
`docker compose exec app uname -s`	Verific sistemul din container. Afișează Linux.
`docker compose exec app sh`	Deschid un shell în container. Ies cu exit.
`docker compose down`	Opresc proiectul și păstrez volumul MySQL.

În browser deschizi **localhost:3000** sau **localhost:3001** dacă ai configurat APP_PORT=3001. **.env** rămâne local, iar modelul de configurare pentru GitHub este **.env.example**.

Referințe Docker WSL 2 Pornirea cu Compose Comenzi în container